Logika Scoring NAP

Sistem CAT Psikotes (psiko-cat) · Nilai Akhir Psikotes (NAP) · Ditelusuri dari kode sumber per 8 Oktober 2026
Sumber: `lib/scoring/*.ts`, acuan: `TRESHOLD PENILAIAN PSIKO POLRI.py`

---

1. Ringkasan

NAP berskala 0–100 dan dibentuk dari tiga sub-tes dengan bobot tetap:

```
NAP = Kontribusi Kecerdasan (maks 60) + Kontribusi Kepribadian (maks 20) + Kontribusi Kecermatan (maks 20)
```

Peserta **Memenuhi Syarat (MS)** bila NAP ≥ 61 **dan** tidak ada sub-tes dengan nilai murni ≤ 40. Selain itu peserta **Tidak Memenuhi Syarat (TMS / gugur)**.

Sub-tes	Jumlah soal	Durasi	Bobot	Kontribusi maks
Kecerdasan	100	90 menit	60%	60
Kepribadian	100	60 menit	20%	20
Kecermatan	500 (10 lajur)	10 menit	20%	20
Total	**700**	**160 menit**	**100%**	**100**

2. Alur Perhitungan

1. **Pemicu.** Perhitungan dijalankan dari tiga titik:
   - Tombol *Hitung nilai NAP* / *Paksa submit* di halaman sesi (`app/test/[sessionId]/page.tsx`).
   - API `POST /api/sessions/[id]/calculate`.
   - Admin recalculate (`app/api/admin/recalculate`) dengan `force = true` untuk menghitung ulang sesi yang sudah final.
2. **Runner.** `runSessionCalculate()` di `lib/scoring/runner.ts` mengambil semua `module_sessions`, mengurutkan per `sequence_order`, lalu memanggil scorer per modul:
   - Kecerdasan & Kepribadian: membaca tabel `answers` + `questions.scoring_rule`.
   - Kecermatan: membaca `kecermatan_logs` (diurutkan per `clicked_at_ms`, satu log per soal), lalu diringkas per lajur.
3. **Penggabungan.** Hasil tiap modul digabung di `lib/scoring/nap.ts`:
   - Sesi penuh (lebih dari 1 modul): `calculateNAP()`.
   - Sesi standalone (1 modul): `calculateSingleModuleResult()`.
4. **Penyimpanan.** Semua hasil ditulis dalam satu transaksi oleh RPC Postgres `finalize_test_session` (mengunci baris sesi dan mengecek ulang status final). Definisi SQL RPC ini hanya ada di database Supabase, tidak tersimpan di repo.

3. Sub-tes Kecerdasan

File: `lib/scoring/kecerdasan.ts`

Pilihan ganda A–E, skor dikotomis (benar = 1, salah/kosong = 0). Sebagian butir meminta dua jawaban; kuncinya disimpan sebagai string kanonis (mis. `"AC"`) dan harus sama persis dengan `selected_key`.

```
Nilai Murni      = (jumlah_benar / 100) × 100
Kontribusi NAP   = jumlah_benar × 0.6          (maks 60)
Gugur            = Nilai Murni ≤ 40
```

4. Sub-tes Kepribadian

File: `lib/scoring/kepribadian.ts`

Skala Likert 5 opsi. Tidak ada jawaban salah; tiap opsi punya bobot 0.04–0.20 yang disimpan di `scoring_rule.weights`. Soal yang tidak dijawab bernilai 0.

Polaritas item	A	B	C	D	E
Favorable (positif)	0.20	0.15	0.10	0.05	0.04
Unfavorable (negatif)	0.04	0.05	0.10	0.15	0.20

```
Poin Akumulasi   = Σ bobot opsi terpilih        (0–20)
Nilai Murni      = (Poin Akumulasi / 20) × 100
Kontribusi NAP   = Poin Akumulasi               (maks 20)
Gugur            = Nilai Murni ≤ 40   (setara Poin Akumulasi ≤ 8)
```

5. Sub-tes Kecermatan

File: `lib/scoring/kecermatan.ts`

10 lajur × 50 soal, batas waktu 60 detik per lajur, tidak bisa kembali ke lajur sebelumnya. Input berupa statistik per lajur: `total_klik` dan `total_benar`. Data divalidasi: harus tepat 10 lajur unik, klik 0–50 per lajur, benar ≤ klik. Bila tidak valid, perhitungan dihentikan dengan error.

Tiga indeks (skala 0–100)

```
Ke (Kecepatan)   = (total_klik / 500) × 100
Kt (Ketelitian)  = (total_benar / total_klik) × 100      (0 bila total_klik = 0)
SD               = √( Σ (x_i − x̄)² / 10 )           x_i = klik di lajur ke-i
Kh (Ketahanan)   = max(0, 100 − (SD / 15) × 100)
```

Kh = 100 bila jumlah klik per lajur rata (SD = 0), turun linear, dan menjadi 0 saat SD ≥ 15. Angka 15 (`MAX_SD`) ditetapkan di kode; dokumen acuan tidak menyebut angka konversinya.

Nilai murni dan kontribusi

```
Nilai Murni      = Ke × 0.30 + Kt × 0.40 + Kh × 0.30
Kontribusi NAP   = Nilai Murni × 0.20                    (maks 20)
Gugur            = Nilai Murni ≤ 40
```

6. Aturan Kelulusan

6.1 Sesi penuh — `calculateNAP()`

Pengecekan dilakukan berurutan. Kondisi pertama yang terpenuhi menjadi `disqualified_reason`:

1. Nilai murni Kecerdasan ≤ 40 → gugur (Kurang Sekali).
2. Nilai murni Kepribadian ≤ 40 → gugur (Kurang Sekali).
3. Nilai murni Kecermatan ≤ 40 → gugur (Kurang Sekali).
4. NAP < 61 → gugur (di bawah passing grade).
5. Selain itu → lulus, status `COMPLETED`.

Bila sebuah modul tidak ada di sesi, modul itu diisi nilai `NOT_TAKEN`: kontribusi 0, nilai murni 100, tidak menggugurkan.

6.2 Sesi standalone (1 modul) — `calculateSingleModuleResult()`

Ambang NAP 61 tidak dipakai, karena satu modul saja tidak mungkin mencapai 61 poin kontribusi. Sebagai gantinya:

```
nap_score  = Nilai Murni modul
is_passed  = Nilai Murni > 40
predikat   = getNAPPredikat(Nilai Murni)
```

6.3 Predikat — `getNAPPredikat()`

Rentang nilai	Predikat	Status (acuan)
≥ 81	Baik Sekali	MS
61 – <81	Cukup	MS
41 – <61	Kurang	TMS
< 41	Kurang Sekali	TMS

Predikat hanya dihitung untuk sesi standalone (di halaman hasil). Untuk sesi penuh, kolom predikat bernilai `null`.

7. Contoh Perhitungan Sesi Penuh

Komponen	Data peserta	Nilai murni	Kontribusi
Kecerdasan	70 benar dari 100	70.00	42.00
Kepribadian	Σ bobot = 15.00	75.00	15.00
Kecermatan	400 klik, 380 benar; klik per lajur 42, 41, 40, 40, 40, 40, 40, 39, 39, 39	90.21	18.04
NAP			**75.04**

Rincian Kecermatan: Ke = 400/500 × 100 = 80; Kt = 380/400 × 100 = 95; rata-rata klik 40, SD = 0.894, Kh = 100 – 0.894/15 × 100 = 94.04. Nilai murni = 80×0.3 + 95×0.4 + 94.04×0.3 = 90.21.

Hasil: tidak ada nilai murni ≤ 40 dan NAP 75.04 ≥ 61 → **MS, predikat Cukup**.

8. Temuan dari Penelusuran Kode

Belum ada perubahan kode. Temuan #1 dan #2 sudah diverifikasi dengan menjalankan angka di Node.js.

BUG **1. Error pembulatan float di batas NAP 61**

Perkalian dan penjumlahan bilangan desimal di JavaScript tidak selalu presisi. Contoh: 41 benar Kecerdasan (24.6) + Kepribadian 17 + Kecermatan 19.4 dihitung sebagai `60.99999999999999`, sehingga `< 61` dan peserta dinyatakan gugur, padahal seharusnya tepat 61 (MS). Akumulasi bobot Kepribadian juga bergeser (100 × 0.05 = `4.99999999999999`).

Usulan: bulatkan nilai (mis. 2 desimal) sebelum dibandingkan dengan ambang 61 dan 40.

BUG **2. Predikat tidak konsisten untuk nilai 40–41**

Pada sesi standalone, nilai murni 40.5 menghasilkan `is_passed = true` (karena > 40), tetapi predikat "Kurang Sekali" (karena "Kurang" dimulai dari ≥ 41). Nilai Kepribadian dan Kecermatan berupa pecahan, sehingga kasus ini bisa terjadi.

BEDA SPEK **3. Kh ≤ 40 tidak menggugurkan**

Dokumen acuan menyatakan Kh ≤ 40 = gugur otomatis (Kurang Sekali). Kode hanya mengecek nilai murni Kecermatan gabungan, bukan Kh secara terpisah. Kh ≤ 40 setara dengan SD ≥ 9.

CELAH SPEK **4. Rumus Kecermatan bisa dimanipulasi**

Kt dan Kh tidak dipengaruhi jumlah soal yang dikerjakan. Peserta yang hanya menjawab 1 soal per lajur dan semuanya benar mendapat Ke = 2, Kt = 100, Kh = 100, sehingga nilai murni = 70.6 dan lulus. Ini kelemahan rumus di dokumen acuan; kode mengikuti rumus tersebut apa adanya.

CATATAN **5. Pembagi di-hardcode**

Kecerdasan dibagi 100 dan Kepribadian dibagi 20 secara tetap. Bila jumlah soal aktif bukan 100, skor tidak sesuai. Akumulasi Kepribadian tidak dibatasi, sehingga bisa melebihi 20 bila soal lebih dari 100.

CATATAN **6. Belum ada unit test scoring**

Tidak ada test untuk `nap.ts`, `kecerdasan.ts`, `kepribadian.ts`, maupun `kecermatan.ts`. Satu-satunya test di `lib/` adalah `kecermatan-symbols.test.ts`.

9. Peta File

File	Peran
`lib/scoring/runner.ts`	Orkestrasi: ambil data, panggil scorer, gabung, simpan via RPC
`lib/scoring/nap.ts`	`calculateNAP`, `calculateSingleModuleResult`, `getNAPPredikat`
`lib/scoring/kecerdasan.ts`	Scoring dikotomis
`lib/scoring/kepribadian.ts`	Scoring bobot Likert
`lib/scoring/kecermatan.ts`	Indeks Ke, Kt, Kh dan nilai murni
`lib/types/safe-question.ts`	Tipe `scoring_rule` per modul
`app/test/[sessionId]/result/page.tsx`	Tampilan hasil; ambang 61 (penuh) / 40 (standalone)
`app/api/admin/recalculate/route.ts`	Hitung ulang paksa oleh admin